

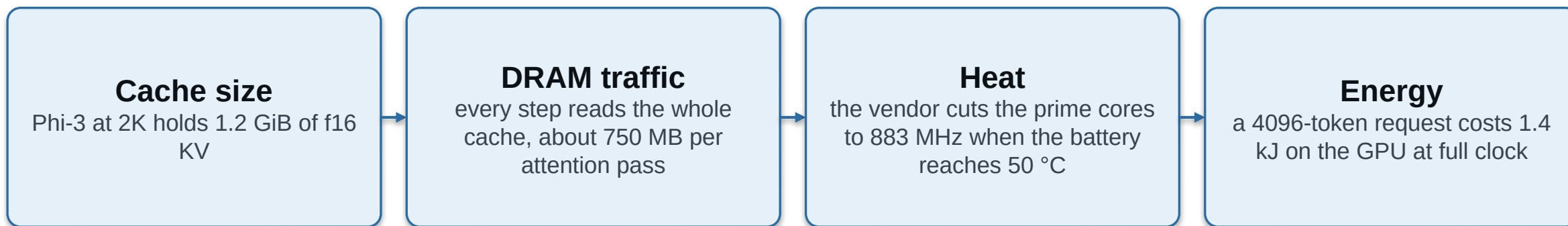
μ KV: Energy- and Thermal-Aware KV-Cache Eviction for LLM Inference on a Phone

The KV cache, not compute, is the wall. The policies that bound it were built for server GPUs, and three properties of a real on-device engine break them.

Md Romyull Islam · Kennesaw State University

OnePlus 15, Snapdragon 8 Elite Gen 5, Adreno 840 · every number measured on the device

The phone is bound by the cache, not the matrix multiply



The four are one chain. Cache size sets traffic, traffic sets power, power sets temperature, temperature sets the throttle, and the throttle sets throughput.

Across 372 on-device cells, live cache correlates with DDR temperature at $r = 0.80$ and with throughput at $r = -0.77$.

So a mobile cache policy has to be judged on quality, throughput, heat and energy at once, on real hardware.

Nine policies, and three engine properties that break them

1 Budgets do not materialize

one cell array serves every head and layer, so a cell is freed only when all of them drop it

2 Scores cost the fast path

reading post-softmax attention forces the FlashAttention-off path

3 Compaction doubles memory

the state API restores survivors into a second full context

None of the three is visible on a server with per-head paged storage.

All three decide what a policy can actually do on a phone, and each is measured on the next three slides.

A nominal budget is not a freed cell

at a matched budget of 1024	live cells	share of the prompt
vanilla, Llama-3.2-1B CPU	9737	100%
µKV	721	7.4%
SnapKV	5929	61%
H2O	6110	63%
TOVA	4091	42%
Ada-KV	3484	36%

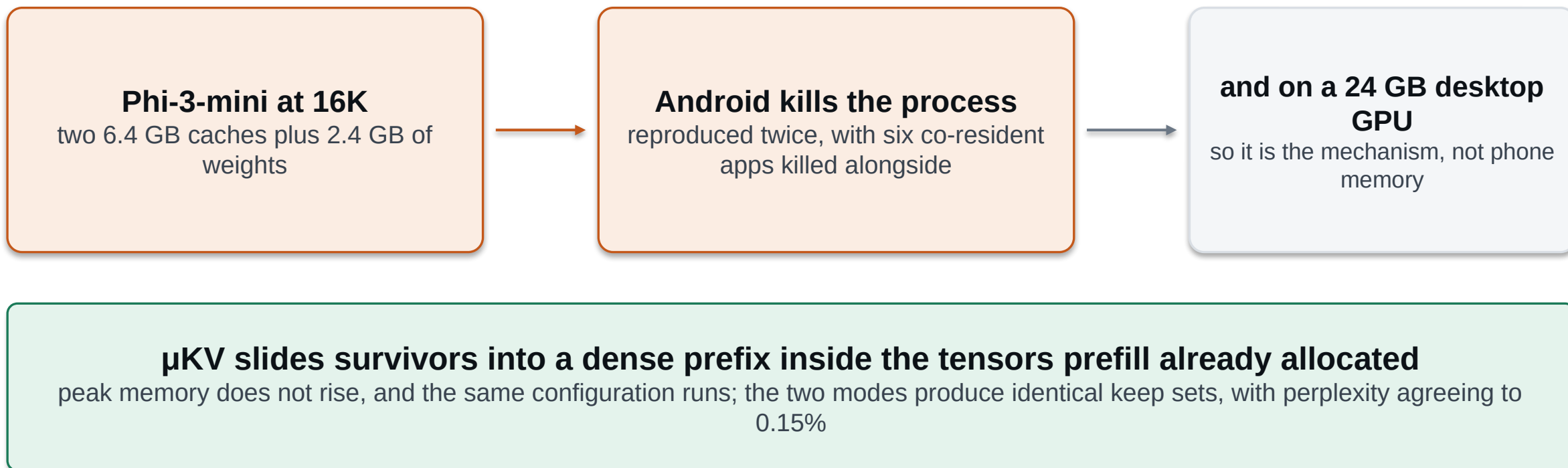
- A cell survives if any head or any layer keeps it, so the per-head unions cover most of the prompt.
- At their own published budgets the gap widens: on 110 LongBench prompts SnapKV at 2048 per head keeps 94% of the cache and H2O at 20% of the prompt keeps 92%.
- SnapKV reports an 8.2 times smaller cache, H2O 5 to 10 times. On this engine they deliver 1.6 to 2.8.
- µKV picks one keep set for every head and layer, which is what lets the cache compact.

Reading attention scores costs more than eviction saves

Llama-3.2-1B, Adreno 840	tok/s	vs full cache
vanilla (full cache)	24.18	1.00
µKV	29.77	1.23
SnapKV	4.76	0.20
TOVA	4.34	0.18
H2O	3.58	0.15
Ada-KV	2.86	0.12

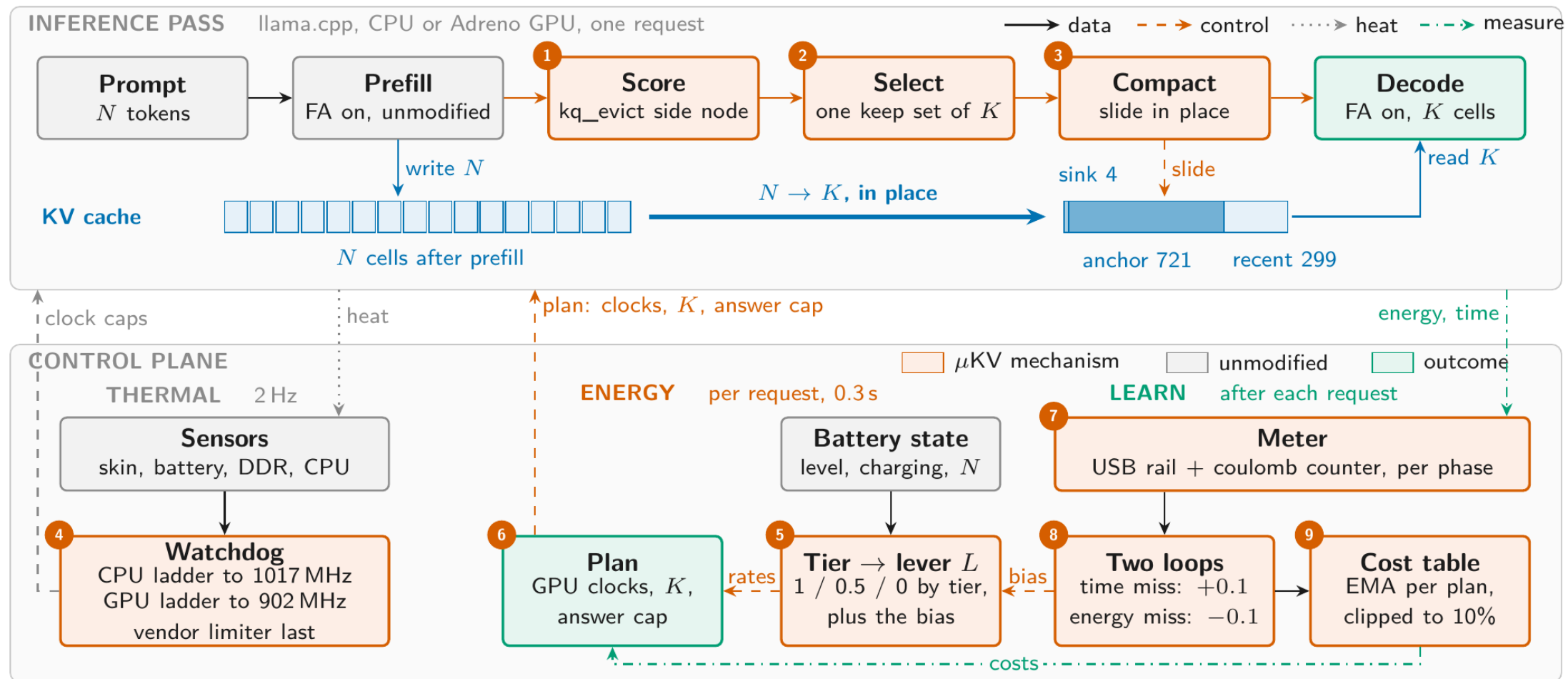
- FlashAttention never writes the attention tensor, so a policy that reads scores cannot use it.
- Every score-reading evictor decodes slower than not evicting at all, 0.12 to 0.30 times the full cache.
- Retention does not predict speed. StreamingLLM keeps 777 cells, fewer than µKV's 723, and still runs at 0.23 because it stays on the slow path in this engine.
- µKV scores inside the FlashAttention graph, so it keeps the signal and the fast path.

The state API needs a second full context



Measured: in-place compaction runs Phi-3 at 16K and reaches 2.64 times the full cache. The round trip runs only at ctx 8192.

The pass on top, the control plane below

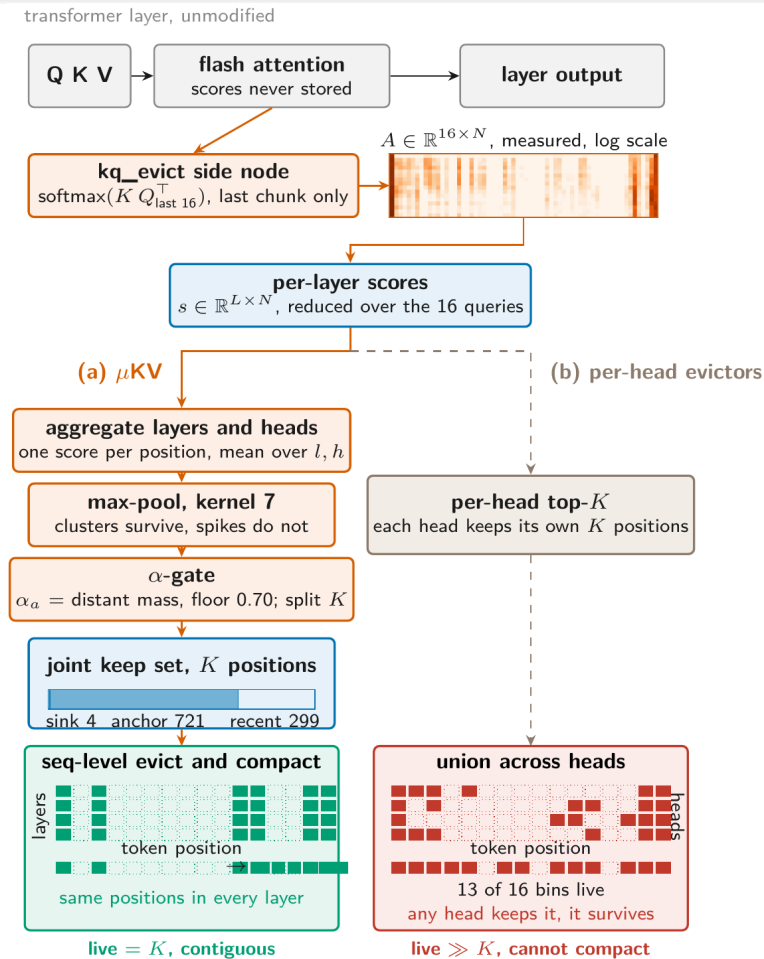


Steps 1 to 3 are the only changes to the inference pass. Steps 4 to 9 are the thermal, energy and learning columns.

Figure 1 of the paper.

One keep set for every head and layer

- The side node computes the real softmax scores for the last 16 queries of the last prefill chunk.
- One score per position: the mean over layers and heads, max-pooled over 7 positions.
- The split α is the share of attention mass outside the recent window, floored at 0.70.
- $K = 1024$ becomes 4 sinks, 721 anchors, 299 recent.



- Per-head keep sets are a union, so 13 of 16 position bins stay live and nothing compacts.
- Heatmap and grids are measured, not drawn: Llama-3.2-1B, layer 8.

9737-token prompt, 4096 generated, cold start, charging off

Llama-3.2-1B, K = 1024	tok/s	wall (s)	live cells	DDR °C	mWh
vanilla (full cache)	5.0	1055	9737	59.8	1033
µKV	24.2	406	721	54.4	391
SnapKV	6.8	902	5929	56.3	818
Ada-KV	6.7	875	3484	57.1	1002
StreamingLLM	6.6	877	777	56.7	1052
H2O	5.8	1006	6110	54.8	1094
TOVA	6.0	947	4091	54.4	1069

4.8×

the full cache's throughput

−62%

energy for the same work

7.4%

of the cells kept

−5 °C

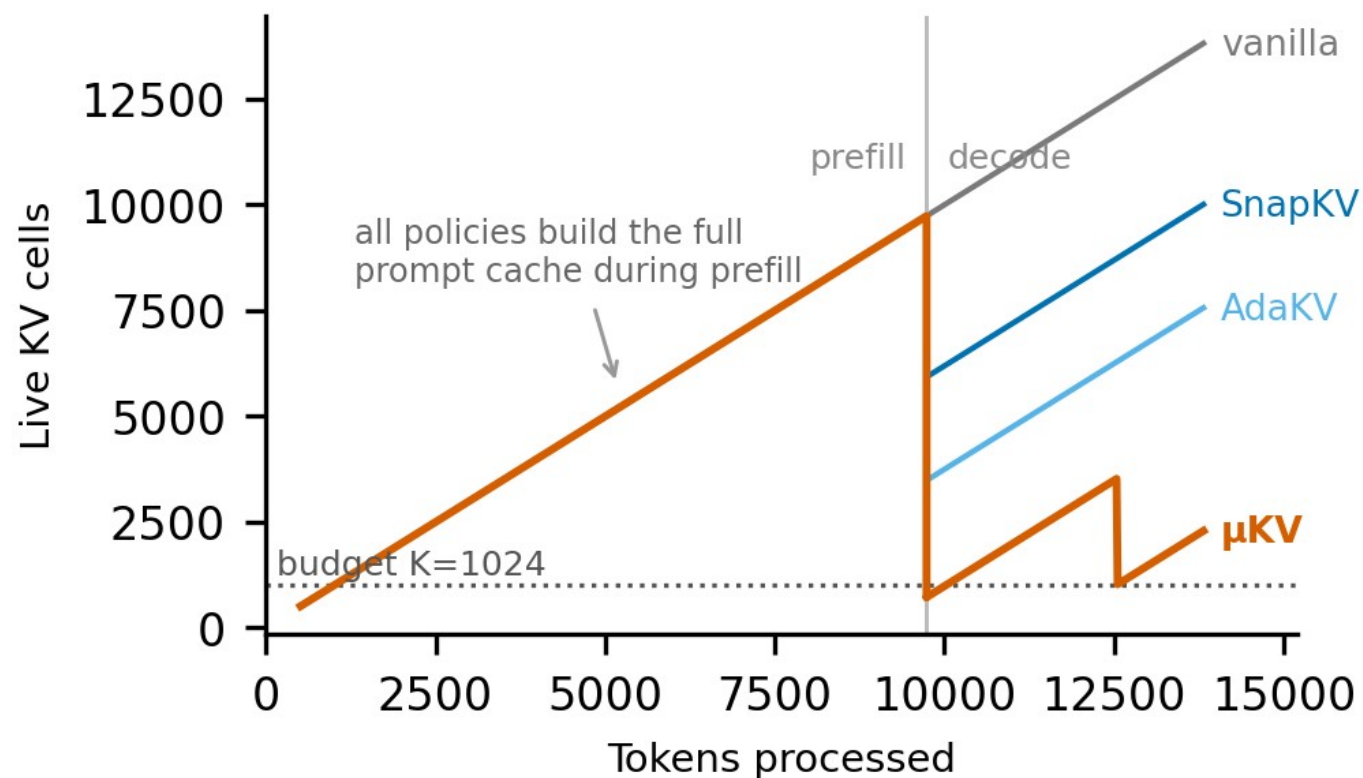
peak DDR

Both policies run the same vendor-clamped 1.6 GHz, so the heat comes from memory traffic, not the clock.

Only µKV both compacts and keeps decode on the fused kernel.

Table 1 of the paper. Three more models are in the paper: Bonsai-8B 4.5x, Phi-3-mini 4.5x, gemma-2-2b 2.4x, all against their own full cache.

Every policy builds the full prompt cache. Only μ KV gives it back



- At the end of prefill μ KV compacts to 721 cells and stays bounded near 2K through decode.
- SnapKV can only drop to 5.9K, because a cell needs every head to release it.
- Vanilla grows without bound.
- This one picture is the paper's argument: the budget a policy asks for is not the cache it frees.

The same workload on the Adreno 840

Llama-3.2-1B	tok/s	dec ×	cells	Phi-3-mini	tok/s	dec ×	cells
vanilla	24.18	1.00	9741	vanilla	5.05	1.00	11157
μKV	29.77	1.23	723	μKV, in place	13.36	2.64	878
μKV, no compaction	25.61	1.06	723	StreamingLLM, own budget	10.60	2.10	2000
best baseline (StreamingLLM)	5.55	0.23	777	SnapKV, gate promoted	5.80	1.15	10862

- In-place compaction runs Phi-3 at 16K, where the state-API round trip is killed. That is the configuration the paper's third break describes.
- Phi-3 needs a driver gate: the FlashAttention-off capture is numerically broken at head dimension 96, and uncorrected runs decode at full speed while emitting 18 to 44% corrupted tokens. The gate promotes SnapKV and Ada-KV to the in-graph side node and refuses H2O and TOVA.

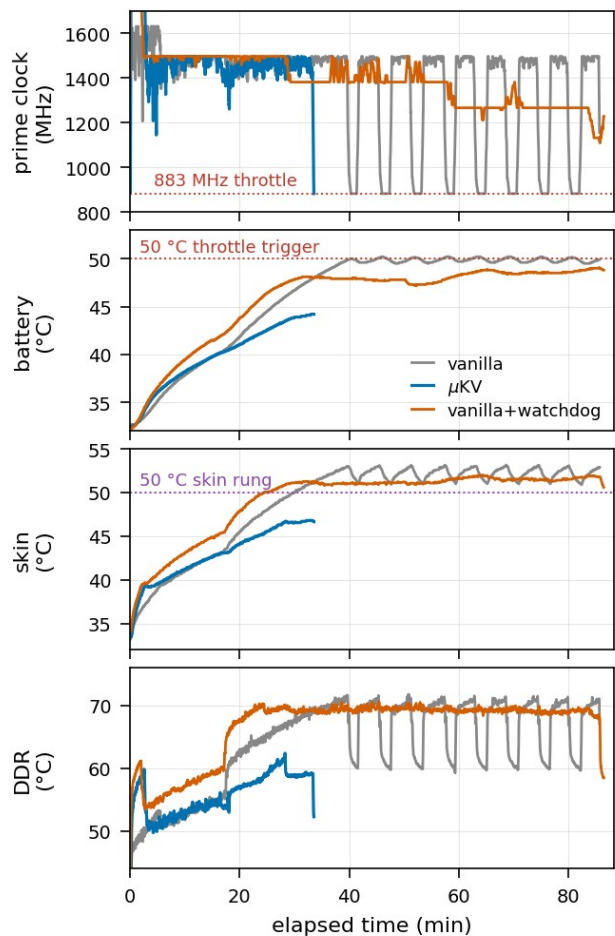
Retrieval and long-context, at each policy's own budget

Policy	needle hits, Phi-3 / Llama / Gemma / Bonsai	cells attended	LongBench F1	cache held
vanilla (full cache)	14 / 13 / 11 / 14	4858	36.9	100%
µKV	14 / 13 / 11 / 14	774	37.0	14%
SnapKV	14 / 12 / 11 / 14	4237	38.0	94%
Ada-KV	14 / 12 / 11 / 14	3361	37.8	71%
TOVA	14 / 12 / 11 / 14	3670	37.4	77%
H2O	14 / 8 / 10 / 13	4212	36.2	92%
StreamingLLM	3 / 3 / 2 / 3	890	35.0	33%

µKV matches the full cache on both benchmarks while holding a seventh of the cache. The others buy about a point of F1 by holding 71 to 94% of it.

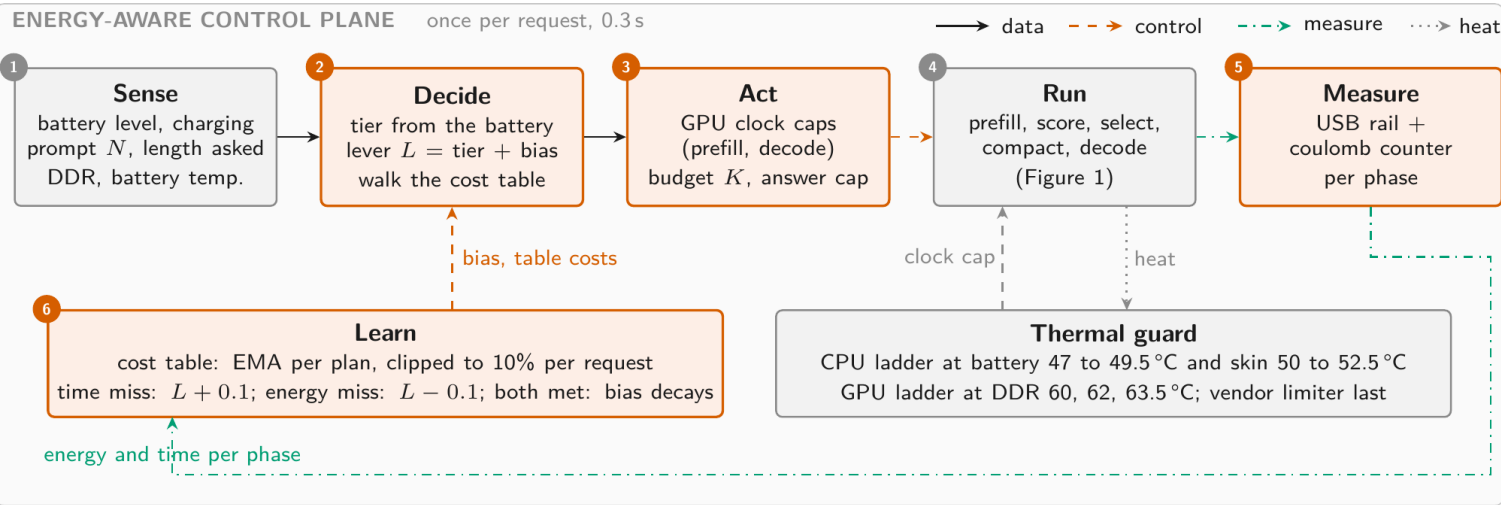
StreamingLLM fails retrieval everywhere: its window evicts exactly the middle of the context, which is where a buried fact lives.

Two levers against the vendor's cliff



- The full cache on Bonsai-8B runs 85.6 minutes and drives the battery to 50.2 °C. At 50 °C the vendor deep-throttles both clusters: 17 dips, 923 s at 883 MHz.
- μKV finishes in 33.2 minutes at a 44.2 °C peak, before the trigger. That is the cache lever.
- Given the watchdog as an ablation, the full cache's battery rise flattens from 0.48 to 0.05 °C per minute and settles at 49.1 °C. The prime cores never reach 883 MHz. That is the clock lever.
- The watchdog belongs to μKV and is never given to a baseline.
- On the GPU the ladder gives 1289 against 1390 J and a DDR peak of 75 against 90 °C, for 1% more time.
- Negative result: a cache budget tied to temperature does not cap peak heat. Cache controls throughput and energy; the clock caps heat.

One lever from the battery, two loops on model error

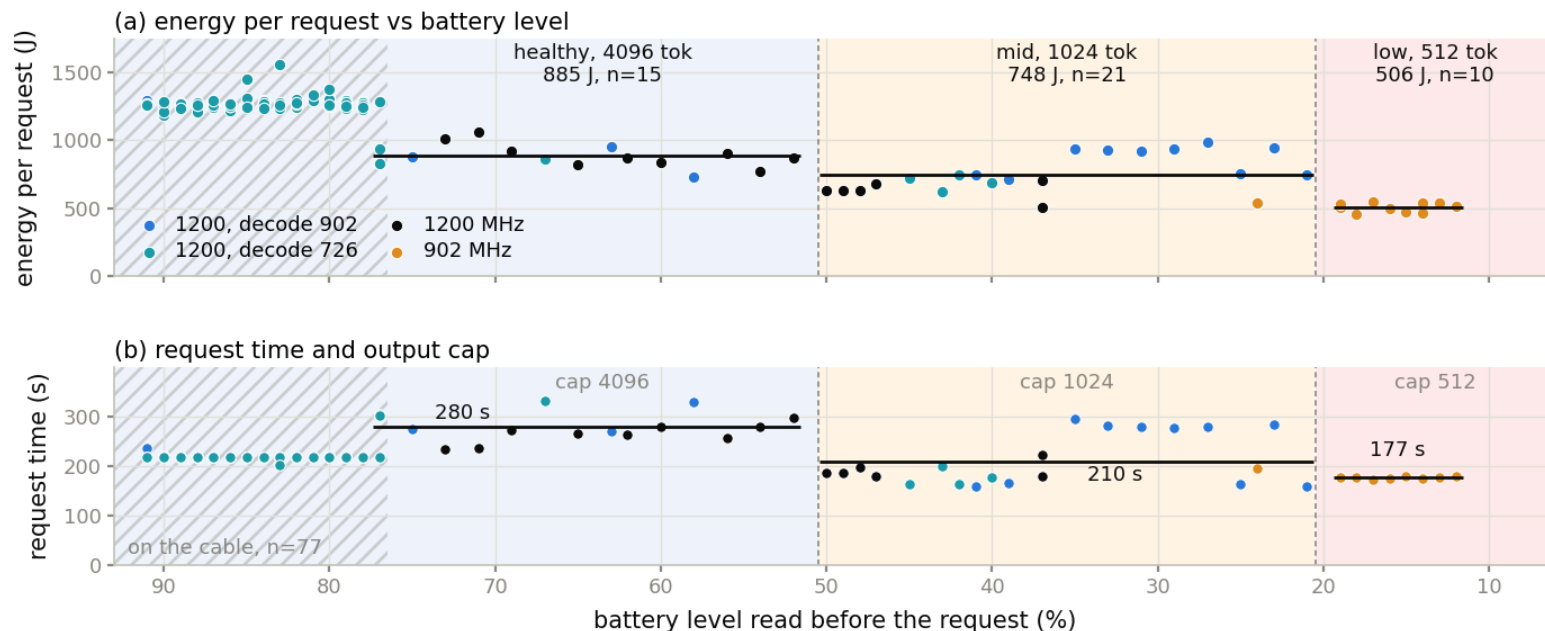


tier	J	s	vs healthy
healthy, cap 4096	885	280	—
mid, cap 1024	748	210	-15%
low, cap 512	506	177	-43%

- 123 requests, 91% to 11%, charging off, nothing forced.
- It switched on the first request past each threshold, 50% and 19%.
- A table seeded on the cable over-predicted by 36%; the clipped learning fixed it in three requests.

Figure 3 and Table 6 of the paper. A decision costs 0.3 s on the phone.

It switches on the real battery, and it saves



- On the battery the phone is a different machine. The same healthy request costs 885 J instead of 1274 on the cable.
- The OEM caps prefill at 726 MHz within a minute of every request, so the answer cap and the prefill cap carry the saving.
- Mean prediction error over the run: 8.9% on the battery, 3.7% on the cable.

Would a learned policy beat the rule? Three experiments said no

1 Simulator

bandit over 72 values and a Q-learner over 576, 60 episodes and 6 seeds

13 / 15

free pulls chose the rule's arm

2 Real cells, offline

a bandit fitted on logged requests, bootstrap for the best arm

15.7 kJ

spent to learn it

3 On the phone

24 real requests, ϵ -greedy over three GPU clocks

149 min

of phone time

Energy and time per plan do not depend on the state of charge; only the weighting does, and that is a design choice. So the table learns and the rule decides.

The negative results are load-bearing

- Cache size is not a temperature actuator. Halving K under co-load left peak DDR unchanged, and on a cool phone a smaller cache runs hotter at peak.
- The per-head budget gate we built is idle. Its candidate pool holds 96.5 to 99.8% of the prompt, and bypassing it leaves the keep set identical, so it is removed.
- A learned policy does not beat the rule, and we report what learning cost.
- Eviction still costs verbatim recall of the spans it drops, even though prediction and retrieval are preserved.
- The 1-bit kernels return non-finite logits on this GPU, so Bonsai's phone numbers are CPU numbers, and every backend now needs an output-validity check.

μKV in four lines

- Three engine properties break server eviction policies on a phone: shared cells, the fast path, and the second context.
- μKV answers all three: scores from inside the FlashAttention graph, one keep set for every head and layer, compaction in place.
- 4.8 times the full cache's decode throughput on the CPU and 1.23 on the GPU, at 7% of the cells and 62% less energy, with full-cache retrieval and LongBench.
- Around it, a clock watchdog keeps the vendor throttle unreachable and a per-request scheduler spends the battery by its level: 15% and 43% less per request at the lower tiers.